

Projektowanie aplikacji mobilnych

Ćwiczenia #2 – Architektura aplikacji mobilnych

Grupa:

Ilias Abdeljalil (40028)

1. Wygenerować nowy projekt w Android Studio lub XCode (0.5pkt)

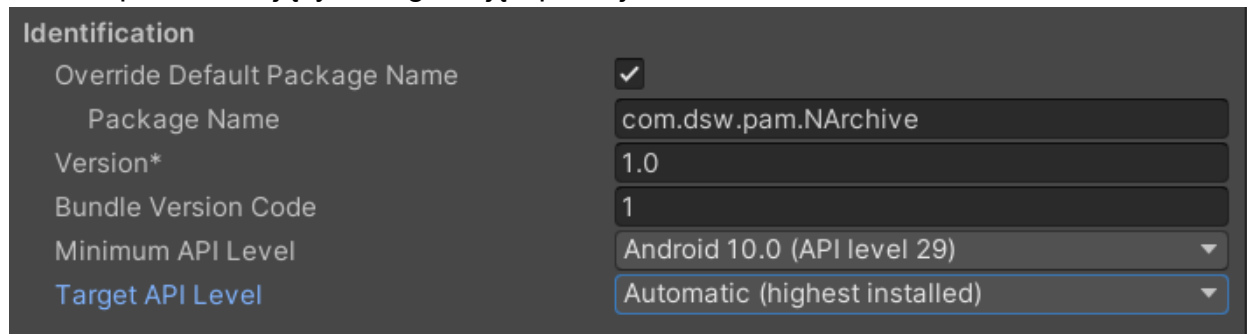
Wymagania Android:

- Min SDK = API 29 (Android 10)
- Build Configuration Language = Kotlin DSL
- Package name = com.dsw.pam.<nazwa-aplikacji>
- Language: Kotlin
- UI: Jetpack Compose (XML też może być jeśli się zgodziłem)

Wymagania iOS:

- Bundle Identifier = com.dsw.pam.<nazwa-aplikacji>
- Interface: SwiftUI
- Language: Swift

Screen potwierdzający konfigurację aplikacji:



The screenshot shows the 'Identification' screen in Android Studio. It contains the following fields and values:

Field	Value
Override Default Package Name	☒
Package Name	com.dsw.pam.NArchive
Version*	1.0
Bundle Version Code	1
Minimum API Level	Android 10.0 (API level 29)
Target API Level	Automatic (highest installed)

2. Utworzyć puste repozytorium na GitHub i wrzucić projekt bez żadnych zmian po wygenerowaniu (0.5pkt)

Po utworzeniu repozytorium, proszę dodać pozostałych członków grupy.

Link: <https://github.com/SeCalfa/NArchive.git>

Screen repozytorium:



3. Zaimplementować jeden z ekranów aplikacji zgodnie ze wzorcem MVVM (3pkt)

Screen przedstawiający zaimplementowany ekran (uruchomiony na urządzeniu lub emulatorze):

NArchive



Remove

Add



Screen kodu odpowiadającego za “Model” (może być więcej niż jeden przykład):

```
Ссылка: 2
public class ArchiveModel
{
    Ссылка: 4
    public List<Folder> Folders { get; set; } = new List<Folder>();
}

Ссылка: 3
public class Folder
{
    Ссылка: 0
    public List<TextDocument> TextDocuments { get; set; } = new List<TextDocument> { };
    Ссылка: 0
    public List<ListDocument> ListDocuments { get; set; } = new List<ListDocument> { };
}

Ссылка: 2
public class TextDocument
{
    Ссылка: 0
    public string Title { get; set; }
    Ссылка: 0
    public string Text { get; set; }
}

Ссылка: 2
public class ListDocument
{
    Ссылка: 0
    public string Title { get; set; }
    Ссылка: 0
    public List<string> Elements { get; set; }
}
```

Screen kodu odpowiadającego za “View” (Activity/Fragment/Composable/...):

```
public class ArchiveView : MonoBehaviour
{
    [SerializeField]
    private TextMeshProUGUI foldersCountText;
    [SerializeField]
    private ArchiveViewModel viewModel;

    Сообщение Unity | Ссылки: 0
    private void Awake()
    {
        viewModel.OnFoldersCountChanged += UpdateFoldersCount;
    }

    Ссылки: 1
    private void UpdateFoldersCount(int count)
    {
        foldersCountText.text = count.ToString();
    }
}
```

Screen kodu odpowiadającego za “ViewModel”:

```
public class ArchiveViewModel : MonoBehaviour
{
    public event Action<int> OnFoldersCountChanged;

    private ArchiveModel archiveModel;

    Сообщение Unity | Ссылки: 0
    private void Awake()
    {
        archiveModel = new ArchiveModel();
    }

    Ссылки: 2
    private void UpdateFoldersCount()
    {
        OnFoldersCountChanged.Invoke(archiveModel.Folders.Count);
    }

    Ссылки: 1
    public void AddFolder()
    {
        archiveModel.Folders.Add(new Folder());

        UpdateFoldersCount();
    }

    Ссылки: 1
    public void RemoveFolder()
    {
        if(archiveModel.Folders.Count == 0)
            return;

        archiveModel.Folders.RemoveAt(0);
        UpdateFoldersCount();
    }
}
```

4. Zaimplementować jeden z ekranów aplikacji zgodnie ze wzorcem MVI (2pkt)

Może być to ten sam ekran (na innym branchu) lub kolejny ekran.

Screen kodu odpowiadającego "Model" (może być więcej niż jeden przykład):

Screen kodu odpowiadającego "View" (Activity/Fragment/Composable/...):

Screen kodu odpowiadającego "ViewModel":

Screen kodu odpowiadającego "Intent":